

Problem Set Solutions: Seminar 06

Heaps II and Amortized Analysis
MIT-style solution format

Algorithms and Data Structures I

June 29, 2026

Course: Algorithms and Data Structures I
Topic: Binomial heaps, k -ary heaps, Fibonacci heaps, amortized analysis
Style: Full problem-set solutions with algorithms, proofs, and complexity bounds

Why this matters. Classical heaps give logarithmic worst-case guarantees, but many algorithms need a sharper accounting: some operations are expensive only because previous cheap operations created deferred work. Binomial heaps are binary counters in disguise; Fibonacci heaps delay consolidation to make `DECREASEKEY` cheap; dynamic arrays and two-stack queues are correct only if one understands why rare rebuilding steps are paid for by many ordinary operations. The central idea of this sheet is to distinguish worst-case time from aggregate and amortized time.

Contents

Problem 1. Number of vertices at depth k in a binomial tree	2
Problem 2. How long would <code>siftDown</code> take in a binomial heap?	2
Problem 3. Total time of n insertions into an empty binomial heap	3
Problem 4. Binomial-heap power with worst-case $O(1)$ insertion	4
Problem 5. k -ary heaps	5
Problem 6. Choosing the arity when insertions dominate extractions	6
Problem 7. Dijkstra's algorithm with Fibonacci heaps	7
Problem 8. Stack with <code>multipop</code>	7
Problem 9. Queue implemented with two stacks: potential method	8
Problem 10. Deque based on a dynamic array	9
Problem 11. Average cost of binary increments	9
Problem 12. Turning <code>heapify</code> into worst-case $O(1)$ insertion	10
Problem 13. Dynamic array push and pop: potential method	11

Conventions. A binomial tree of order r is denoted by B_r and has 2^r vertices. Its root has children that are roots of $B_0, B_1, \dots, B_{r-1}$. Heap size is denoted by N when it is not the same as the problem parameter n . Logs are base 2 unless stated otherwise. For amortized analysis, the amortized cost is

$$\hat{c}_i = c_i + \Phi_i - \Phi_{i-1},$$

where c_i is the actual cost and Φ is a nonnegative potential.

Problem 1. Number of vertices at depth k in a binomial tree

Determine the number of vertices at depth k in the binomial tree B_n .

Solution.

Answer. The number of vertices at depth k in B_n is

$$\binom{n}{k}.$$

In particular, the depth ranges from 0 to n .

Correctness. We prove the formula by induction on n . The tree B_0 consists of one vertex, so the numbers of vertices by depths are (1), which agrees with $\binom{0}{0} = 1$.

The binomial tree B_n is obtained by linking two copies of B_{n-1} : one root becomes the root of B_n , and the other copy becomes a child of that root. Therefore the number $D(n, k)$ of vertices at depth k satisfies

$$D(n, k) = D(n-1, k) + D(n-1, k-1).$$

The first term comes from the copy whose root remains the root. The second term comes from the attached copy, whose every depth increases by 1. By the induction hypothesis,

$$D(n, k) = \binom{n-1}{k} + \binom{n-1}{k-1} = \binom{n}{k},$$

by Pascal's identity. This proves the claim for all n and k .

Running time. This is a structural counting argument; no algorithmic computation is needed. If all values for fixed n are required, they can be generated in $O(n)$ time using Pascal's recurrence.

Problem 2. How long would siftDown take in a binomial heap?

Estimate the running time of a SIFTDOWN operation inside a binomial heap.

Solution.

Idea. A binomial tree has logarithmic height, but a node can have logarithmically many children. Thus moving a key down one level is not necessarily constant-time if we must scan all children to find the best one.

Answer. In a standard binomial tree on N vertices, a straightforward SIFTDOWN costs

$$O(\log^2 N).$$

Equivalently, inside a binomial tree B_r it costs $O(r^2)$.

Correctness. Consider a node in B_r . The root has r children. If the heap property is violated and we want to move a large key downward in a min-heap, we must choose the smallest child among those children. This costs $O(r)$ comparisons at the root. After one swap, the key enters a child subtree of order at most $r - 1$, where the next level costs at most $O(r - 1)$ comparisons, and so on.

Therefore a complete downward movement costs at most

$$r + (r - 1) + \cdots + 1 = O(r^2).$$

Since B_r has 2^r vertices, $r = \log N$, so the cost is $O(\log^2 N)$.

Running time. With extra information, for example if each node maintained its best child under updates, one could reduce the child choice cost. In the standard pointer representation without such auxiliary minima, the natural bound is $O(\log^2 N)$.

Problem 3. Total time of n insertions into an empty binomial heap

An initially empty binomial heap receives n INSERT requests. Prove that the total time is $O(n)$, although some individual insertions require $\Omega(\log n)$ operations.

Solution.

Idea. A binomial heap is a binary counter. Inserting one element is like adding 1: a chain of links corresponds to a chain of carries. Carries are occasionally long, but the total number of carries in the first n increments is linear.

Algorithm. Each INSERT creates a singleton tree B_0 . If there is already a tree of order 0, link the two trees into one B_1 . If there is already a tree of order 1, link again, and so on until an empty order is found.

Correctness. The heap invariant is that for every order r there is at most one binomial tree B_r . Linking two B_r trees produces one B_{r+1} tree and preserves heap order by making the smaller root the parent. Thus after the carry chain stops, every order again has at most one tree, and the root of every tree is a heap-ordered binomial tree. Therefore the data structure remains a valid binomial heap after every insertion.

Amortized analysis. Use potential

$$\Phi = \text{number of trees in the binomial heap.}$$

Suppose an insertion performs t links. Its actual cost is $O(t + 1)$. The operation first adds a singleton tree, increasing Φ by 1, and each link replaces two trees by one tree, decreasing Φ by 1. Hence

$$\Delta\Phi = 1 - t.$$

The amortized cost is

$$O(t + 1) + (1 - t) = O(1),$$

up to a constant scaling of the potential. Summing over n insertions gives total time $O(n)$.

Long individual insertion. If before an insertion the heap size is $2^q - 1$, then the binary representation is q ones. The next insertion causes a carry through all q occupied ranks, so it performs $q = \Omega(\log n)$ links. Thus the worst-case time of a single insertion is logarithmic even though the total time of n insertions is linear.

Problem 4. Binomial-heap power with worst-case $O(1)$ insertion

Using binomial heaps, design a data structure with the same heap operations, but such that every INSERT takes worst-case $O(1)$ time. Hint: use a redundant binary counter whose digits may be 0, 1, or 2.

Solution.

Idea. The expensive part of binomial-heap insertion is a long carry chain. A redundant counter avoids long carry chains by allowing two objects of the same rank. Translated to heaps: keep up to two binomial trees of each order and repair carries locally, not all at once.

Algorithm. Maintain buckets $T_0, T_1, T_2, \dots$, where bucket T_r stores 0, 1, or 2 heap-ordered binomial trees of order r . This is a redundant binary representation of the heap size. We also maintain a pointer to the smallest bucket that currently needs local normalization. A local normalization step is:

If some bucket T_r contains three trees, link two of them into one tree of order $r + 1$ and leave one tree in T_r . If this makes T_{r+1} contain three trees, mark $r + 1$ as the next bucket to normalize.

The standard redundant-counter invariant is maintained: the scheduled local normalizations are processed at a constant rate, so a public operation never performs more than $O(1)$ links caused by insertion. This is the heap analogue of a worst-case constant-time increment in a redundant counter with digits 0, 1, 2.

For heap operations:

- INSERT(x): create a singleton B_0 with key x , put it into T_0 , and perform the constant number of local normalizations required by the redundant-counter schedule.
- FINDMIN: scan the roots of all stored trees and return the smallest root.
- MERGE: add corresponding buckets as in redundant-counter addition and normalize the resulting representation. This costs logarithmic time in the resulting heap size.
- EXTRACTMIN: find the tree whose root is minimum, remove that root, split its children into one tree of each smaller order, and add these trees back to the bucket representation. Then normalize.

Correctness. Each bucket contains heap-ordered binomial trees of the bucket's order. Linking two trees of the same order by the smaller root creates a heap-ordered binomial tree of the next order, so local normalization preserves the represented multiset and preserves heap order. The redundant representation permits two trees of the same order, but this does not affect correctness: the minimum of the whole heap is still the minimum among all tree roots. Removing the minimum root and reinserting its children is exactly the usual binomial-heap EXTRACTMIN argument, except that the result is stored in redundant rather than ordinary binary form. Therefore all operations represent the same priority queue as an ordinary binomial heap.

Running time. The critical guarantee is that INSERT performs only $O(1)$ local counter operations and hence worst-case $O(1)$ tree links. Operations that must inspect or combine ranks still touch only $O(\log N)$ ranks, because the largest binomial tree order is $O(\log N)$. Thus FINDMIN, MERGE, and EXTRACTMIN remain logarithmic up to the same asymptotic order as in binomial heaps, while insertion becomes worst-case constant.

Problem 5. k -ary heaps

Instead of binary heaps, consider k -ary heaps, where each internal vertex has k children except possibly near the last level. Implement SIFTUP, SIFTDOWN, and the classical heap operations. Estimate their running times.

Solution.

Idea. A larger branching factor makes the heap shallower, so upward movement becomes cheaper. Downward movement, however, must inspect up to k children per level.

Algorithm. Use a 0-indexed array. The children of vertex i are

$$ki + 1, ki + 2, \dots, ki + k,$$

if these indices are less than the heap size. The parent of $i > 0$ is

$$\left\lfloor \frac{i-1}{k} \right\rfloor.$$

siftUp. While $i > 0$ and the key at i is smaller than the key at its parent, swap i with its parent and continue from the parent.

siftDown. At vertex i , find the smallest among i and all existing children of i . If a child is smaller than i , swap with the smallest child and continue from that child. Otherwise stop.

The standard operations are then:

- GETMIN: return the root, $O(1)$.
- INSERT: append the new element and run SIFTUP.
- EXTRACTMIN: swap the root with the last element, delete the last element, and run SIFTDOWN from the root.
- DECREASEKEY: decrease the stored key and run SIFTUP.
- HEAPIFY: run SIFTDOWN from the last internal vertex down to the root.

Correctness. The array representation is exactly the level-order traversal of a complete k -ary tree. SIFTUP only swaps a vertex with its parent when the heap property is violated on that edge; all other edges remain valid. When the element stops, its parent is no larger than it, and all child subtrees were never invalidated. Thus the heap property is restored. SIFTDOWN is symmetric: if the root of a subtree is too large, swapping it with the smallest child fixes the root edge and moves the only possible violation one level lower. When no child is smaller, the whole affected subtree is a heap. Therefore the classical operations implemented through these routines are correct.

Running time. The height of a k -ary heap on N elements is $O(\log_k N)$. Therefore SIFTUP, INSERT, and DECREASEKEY take $O(\log_k N)$. At each level, SIFTDOWN scans up to k children, hence it takes $O(k \log_k N)$. Thus EXTRACTMIN takes $O(k \log_k N)$ and GETMIN takes $O(1)$. Bottom-up HEAPIFY takes $O(N)$ time, since the total height over all vertices is $O(N/(k-1))$ and each downward level costs $O(k)$, giving $O(kN/(k-1)) = O(N)$ for $k \geq 2$.

Problem 6. Choosing the arity when insertions dominate extractions

It is known in advance that a heap will receive n^a EXTRACTMIN requests and n^b INSERT requests, in arbitrary order, where $0 < a < b$ are fixed constants. Prove that for every n one can choose k so that all requests can be processed by a k -ary heap in $O(n^b)$ total time.

Solution.

Idea. A k -ary heap trades cheaper insertions for more expensive extractions. Since there are many more insertions than extractions, choose k large enough to make the heap height constant, but not so large that the few extractions become too expensive.

Algorithm. Choose

$$k = \lceil n^{b-a} \rceil.$$

Use a k -ary min-heap.

Correctness. The data structure is an ordinary k -ary heap, so by Problem 5 all heap operations are correct for every fixed choice of k .

Running time. At all times the heap has size at most

$$N \leq n^b.$$

An insertion costs

$$O(\log_k N) = O\left(\frac{\log n^b}{\log n^{b-a}}\right) = O\left(\frac{b}{b-a}\right) = O(1),$$

since a, b are constants and $b - a > 0$. Therefore all n^b insertions cost $O(n^b)$.

An extraction costs

$$O(k \log_k N) = O\left(n^{b-a} \cdot \frac{b}{b-a}\right) = O(n^{b-a}).$$

There are n^a extractions, so their total cost is

$$n^a \cdot O(n^{b-a}) = O(n^b).$$

The total cost of all requests is therefore $O(n^b)$.

Problem 7. Dijkstra's algorithm with Fibonacci heaps

A Fibonacci heap supports INSERT in $O(1)$, DECREASEKEY in amortized $O(1)$, and EXTRACTMIN in amortized $O(\log n)$. Dijkstra's algorithm performs n insertions, at most $m \in [n-1, n^2]$ decrease-key operations, and n extract-min operations. Prove the resulting complexity and compare it with binary heaps.

Solution.

Answer. With Fibonacci heaps, Dijkstra's algorithm runs in

$$O(m + n \log n).$$

With binary heaps, the standard bound is

$$O((m + n) \log n) = O(m \log n)$$

when $m \geq n - 1$.

Correctness. This question concerns only the cost of the priority queue operations. Fibonacci heaps implement the same abstract operations required by Dijkstra's algorithm: insertion of vertices, key decreases after relaxations, and repeated extraction of the vertex with smallest tentative distance. Therefore replacing the priority queue implementation does not change Dijkstra's correctness proof.

Running time. The operation counts give

$$n \cdot O(1) + m \cdot O^*(1) + n \cdot O^*(\log n) = O(m + n \log n),$$

where O^* denotes amortized time.

For a binary heap, INSERT, DECREASEKEY, and EXTRACTMIN all cost $O(\log n)$, so the corresponding bound is

$$O(n \log n + m \log n + n \log n) = O((m + n) \log n).$$

Since $m \geq n - 1$, this is $O(m \log n)$.

Comparison. For sparse graphs with $m = \Theta(n)$, both bounds are $O(n \log n)$. For dense graphs with $m = \Theta(n^2)$, Fibonacci heaps give $O(n^2)$, while binary heaps give $O(n^2 \log n)$. The improvement comes exactly from making the many DECREASEKEY operations amortized constant time.

Problem 8. Stack with multipop

A stack supports ordinary PUSH and POP, and also MULTIPOP(k), which removes the top k elements if they exist. Show that the amortized complexity of all operations is $O(1)$ under the usual singly linked-list implementation. Explain why it is false that every operation has actual $O(1)$ time.

Solution.

Idea. Even if one MULTIPOP removes many elements, each removed element must have been previously inserted and can be charged to its own insertion.

Amortized analysis. Use the aggregate method. Consider any sequence of operations. Let P be the number of pushes. Every element pushed onto the stack is physically popped at most once: either by POP or as one of the removed elements of some MULTIPOP operation. Hence the total number of node deletions over the entire sequence is at most P .

The total cost is therefore

$$O(\text{number of operations}) + O(\text{number of deleted nodes}) \leq O(q) + O(P) = O(q),$$

where q is the total number of operations and $P \leq q$. Thus the amortized cost per operation is $O(1)$.

Why not worst-case $O(1)$? Take a stack containing N elements and call MULTIPOP(N). The operation removes N nodes and therefore takes $\Theta(N)$ actual time. Hence the amortized bound is constant, but the worst-case time of a single operation is not.

Problem 9. Queue implemented with two stacks: potential method

Prove using the potential method that a queue implemented with two stacks has amortized $O(1)$ time per operation.

Solution.

Algorithm. Maintain two stacks: S_{in} and S_{out} .

- ENQUEUE(x): push x onto S_{in} .
- DEQUEUE: if S_{out} is nonempty, pop from it. If it is empty, move all elements from S_{in} to S_{out} one by one, then pop from S_{out} .

Potential. Let

$$\Phi = |S_{\text{in}}|.$$

This potential is always nonnegative and initially zero.

Amortized analysis. An enqueue has actual cost 1 and increases Φ by 1, so its amortized cost is

$$1 + 1 = 2.$$

A dequeue with nonempty S_{out} has actual cost 1 and does not change S_{in} , so its amortized cost is 1.

Now suppose S_{out} is empty and $|S_{\text{in}}| = m$. The operation moves m elements and then pops one, so its actual cost is $m + 1$. The potential decreases from m to 0, hence

$$\hat{c} = (m + 1) + (0 - m) = 1.$$

Thus every operation has amortized $O(1)$ cost.

Correctness. S_{in} stores newly enqueued elements in reverse queue order, while S_{out} stores the older elements with the next queue front on top. If S_{out} is empty, reversing S_{in} by moving it into S_{out} places the oldest element on top. Hence every dequeue returns exactly the oldest currently stored element.

Problem 10. Deque based on a dynamic array

Implement a deque, i.e. a dynamic array that supports insertion and deletion at both the beginning and the end. The amortized running time of every operation should be $O(1)$.

Solution.

Idea. Use a circular buffer. The logical first element need not be stored at physical index 0; it is stored at index h , and logical index j is stored at $(h + j) \bmod C$, where C is the capacity.

Algorithm. Maintain an array of capacity C , a size s , and a head index h . The back position is $(h + s) \bmod C$.

- **PUSH_FRONT**(x): if $s = C$, resize to capacity $2C$. Then set $h \leftarrow (h - 1) \bmod C$, write x at h , and increment s .
- **PUSH_BACK**(x): if $s = C$, resize to capacity $2C$. Write x at $(h + s) \bmod C$ and increment s .
- **POP_FRONT**: remove the element at h , set $h \leftarrow (h + 1) \bmod C$, and decrement s .
- **POP_BACK**: remove the element at $(h + s - 1) \bmod C$ and decrement s .

If after a deletion we have $s < C/4$, shrink to capacity $C/2$. A resize copies the s logical elements into a new array in order and resets $h = 0$.

Correctness. The representation invariant is: the deque elements in logical order are exactly

$$A[h], A[(h + 1) \bmod C], \dots, A[(h + s - 1) \bmod C].$$

Each operation changes only the appropriate endpoint and updates h and s so that this invariant remains true. During a resize, the same logical sequence is copied into positions $0, 1, \dots, s - 1$, so the represented deque is unchanged.

Amortized analysis. Ignoring resizes, every operation costs $O(1)$. A resize costs $O(s)$, but after doubling the capacity from C to $2C$, at least C further endpoint insertions are required before the next expansion. After shrinking from C to $C/2$ at load below $1/4$, the new load is below $1/2$, and at least $\Omega(C)$ further operations are needed before another resize of comparable size. Thus resize costs can be charged to the operations since the previous resize. The amortized time per operation is $O(1)$.

A formal potential proof is the same as for dynamic arrays in Problem 13, applied to the circular-buffer size s and capacity C .

Problem 11. Average cost of binary increments

The number 0 is written in an n -bit binary representation. We add 1 exactly $2^n - 1$ times. The time of an increment is the number of 1 bits that become 0 bits. Estimate the average complexity. Which operations are cheapest and which are most expensive?

Solution.

Idea. The cost of an increment is the number of trailing ones in the previous value. A bit becomes 0 exactly when all lower bits are also 1 and the increment carries through it.

Answer. The total cost of all $2^n - 1$ increments is

$$2^n - n - 1,$$

so the average cost is

$$\frac{2^n - n - 1}{2^n - 1} < 1,$$

and tends to 1 as $n \rightarrow \infty$.

Correctness. Index bits from 0 for the least significant bit to $n - 1$ for the most significant bit. Bit j changes from 1 to 0 exactly when the previous number is congruent to $2^{j+1} - 1$ modulo 2^{j+1} . During the increments from 0 to $2^n - 1$, this happens

$$2^{n-j-1} - 1$$

times for bit j : the final all-ones number is reached but not incremented further, so the last possible overflow event is not counted.

Therefore the total number of $1 \rightarrow 0$ bit changes is

$$\sum_{j=0}^{n-1} (2^{n-j-1} - 1) = (2^n - 1) - n = 2^n - n - 1.$$

This is exactly the total time by the definition of the cost.

Cheapest operations. An increment from an even number has no trailing ones, so its cost is 0. These are the cheapest operations.

Most expensive operation. The most expensive increment in this sequence is

$$2^{n-1} - 1 \longrightarrow 2^n,$$

which turns the lower $n - 1$ bits from 1 to 0. Its cost is $n - 1$. We do not perform the overflow increment from $2^n - 1$ to 2^{n+1} , so cost n does not occur.

Problem 12. Turning heapify into worst-case $O(1)$ insertion

Suppose we have a heap supporting INSERT, MERGE, and EXTRACTMIN in $O(\log n)$ time, and HEAPIFY in $O(n)$ time. Build from it a heap with the same operations amortized, but with worst-case $O(1)$ insertion.

Solution.

Idea. Do not insert new keys into the real heap immediately. Keep them in an unsorted buffer. Insertion into the buffer is worst-case constant. When a non-insertion operation needs a real heap, rebuild the buffer in one linear HEAPIFY step and merge it into the main heap. The rebuild is paid for by the buffered insertions.

Algorithm. Maintain:

- a main heap H implemented by the given heap structure;
- a buffer list B of newly inserted keys not yet inserted into H .

The operations are:

- INSERT(x): append x to B . This is worst-case $O(1)$.
- FLUSH: if B is nonempty, build a heap H_B from all keys in B using HEAPIFY in $O(|B|)$ time, set $H \leftarrow \text{MERGE}(H, H_B)$, and clear B .
- EXTRACTMIN: first call FLUSH, then call EXTRACTMIN on H .
- MERGE of two wrapped heaps: merge their main heaps and concatenate their buffers. The next flush will incorporate both buffers.
- HEAPIFY on a given set of keys: call the underlying HEAPIFY to create H and start with empty B .

Correctness. At all times, the represented multiset is the disjoint union of the keys in the main heap H and the keys in the buffer B . Insertion appends exactly one new key to this union. Flush replaces all buffered keys by a heap containing the same keys and merges it with H , so it preserves the represented multiset. Therefore, after flushing, EXTRACTMIN on H returns the minimum of all represented keys. Merge and heapify are also correct because they preserve or create exactly the intended multiset of keys.

Amortized analysis. Use potential

$$\Phi = |B|.$$

An insertion costs 1 and increases Φ by 1, so its amortized cost is $O(1)$ and its actual worst-case cost is also $O(1)$.

A flush of $b = |B|$ elements costs $O(b)$ for HEAPIFY plus $O(\log N)$ for merge, and changes the potential by $-b$. Thus the linear rebuilding part has amortized $O(1)$ total contribution; only the underlying logarithmic heap operation remains. Therefore EXTRACTMIN and MERGE keep their usual logarithmic amortized cost, while INSERT is worst-case constant.

Problem 13. Dynamic array push and pop: potential method

Using the potential method, prove that PUSH_BACK and POP_BACK in a dynamic array have amortized $O(1)$ time.

Solution.

Algorithm. Maintain an array with size s and capacity C .

- If PUSH_BACK is called when $s = C$, allocate capacity $2C$, copy all s elements, and then append.
- If POP_BACK makes $s < C/4$, allocate capacity $C/2$ and copy the remaining s elements.

The quarter threshold prevents immediate grow-shrink oscillation.

Potential. Use the standard nonnegative potential

$$\Phi(s, C) = \begin{cases} 2s - C, & s \geq C/2, \\ C/2 - s, & s < C/2. \end{cases}$$

This potential is always nonnegative.

Amortized analysis. We sketch the four relevant cases.

Push without expansion. The actual cost is 1. If the load remains on the same side of $C/2$, the potential changes by at most 2 in absolute value. If it crosses $C/2$, the potential still changes by $O(1)$. Hence the amortized cost is $O(1)$.

Push with expansion. Before expansion, $s = C$. The actual cost is $C + 1$: copy C elements and append one. Before the operation,

$$\Phi_{\text{mold}} = 2C - C = C.$$

After expansion, the new size is $C + 1$ and the new capacity is $2C$, so

$$\Phi_{\text{mnew}} = 2(C + 1) - 2C = 2.$$

Thus

$$\hat{c} = (C + 1) + (2 - C) = 3 = O(1).$$

Pop without shrinking. The actual cost is 1. As in the non-expanding push case, the potential changes by only $O(1)$, so the amortized cost is $O(1)$.

Pop with shrinking. Suppose before the pop the size is s and capacity is C , and after deleting one element the new size $s' = s - 1$ satisfies $s' < C/4$. We shrink to new capacity $C' = C/2$ and copy s' elements. The actual cost is $O(s' + 1) = O(C)$, but since $s' < C/4$, before the shrink the potential is approximately

$$\Phi_{\text{mold}} = C/2 - s,$$

which is $\Omega(C)$ near the shrink threshold. After shrinking, the new load is below $1/2$, and

$$\Phi_{\text{mnew}} = C'/2 - s' = C/4 - s' = O(1)$$

at the threshold, or in any case smaller by enough to pay for the copy after constant rescaling of the potential. Therefore the amortized cost is $O(1)$.

Correctness. Expansion and shrinking copy the elements in their original order and only change the capacity. Therefore the represented sequence is unchanged by resizing. `PUSH_BACK` appends one element to the end, and `POP_BACK` removes exactly the last element.

Running time. Every operation has amortized $O(1)$ time. Individual resizing operations can take $\Theta(n)$ actual time, so the guarantee is amortized rather than worst-case.